

Brain Tumor MRI Segmentation Using U-Net

Claudia-Mihaela Cazacu
Faculty of Mathematics and Computer Science
University of Bucharest
Bucharest, Romania
claudia-mihaela.cazacu@s.unibuc.ro

Abstract—This paper presents a deep learning approach for binary segmentation of brain tumors from MRI images using the U-Net architecture. The model is trained to produce a pixel-level mask identifying the tumor region. Multiple training experiments were conducted to evaluate the effect of data augmentation and learning rate on model performance. The best configuration achieved a validation Dice Score of 0.8569 and a validation IoU of 0.7629. The model is deployed through a Streamlit web interface that displays the segmentation result and estimates the tumor area as a percentage of the total scan.

Index Terms—brain tumor, MRI segmentation, U-Net, deep learning, data augmentation, Dice loss

I. INTRODUCTION

Brain tumor detection is a critical task in medical imaging, where early and accurate identification can significantly impact patient outcomes. Manual analysis of MRI scans is time-consuming and subject to human error, making automated segmentation approaches highly valuable.

This project presents a deep-learning solution based on the U-Net architecture [1] for binary segmentation of brain tumors from MRI images. The model is trained in order to produce a pixel-level mask identifying the tumor region, and uses a Streamlit web interface [4] for interactive use. The primary goal is not only to achieve accurate segmentation, but also to systematically evaluate how different training strategies affect model performance.

II. DATASET

The dataset used is BRISC2025, from Kaggle [2]. It consists of MRI brain scans that have binary segmentation masks, where the white pixels indicate the tumor. The data is split into training, validation and test sets, ensuring that the model is evaluated on unseen data throughout the training process.

During preprocessing, the images were resized to 256×256 pixels and normalized to the $[0,1]$ range. Masks were binarized using a threshold of 0.5, converting grayscale mask values to binary labels. Images with no corresponding mask file were excluded from training. All images were loaded in grayscale mode, as MRI scans do not contain color information relevant to the segmentation task.

III. METHODOLOGY

The U-Net architecture [1] was selected for its proven performance in medical image segmentation tasks. It follows an encoder-decoder structure, where the encoder progressively reduces spatial dimensions to extract high-level features, and

the decoder reconstructs the spatial resolution to produce the final segmentation mask. Skip connections between encoder and decoder layers preserve fine-grained spatial information that would otherwise be lost during downsampling. This makes U-Net particularly effective for tasks where the precise localization of boundaries is important, such as tumor segmentation.

The loss function combines Binary Cross-Entropy (BCE) and Dice Loss. BCE penalizes incorrect pixel-level predictions, while Dice Loss directly optimizes the overlap between predicted and ground truth masks. This combination is particularly effective in medical segmentation, where tumor regions are significantly smaller than the background, creating class imbalance. Using Dice Loss alone can be unstable during early training, while BCE alone does not sufficiently reward correct overlap, making the combined approach a well-established choice for this type of task.

IV. TRAINING

One of the biggest challenges I had to face during the making of this project was deeply understanding the Machine Learning logic when it comes to choosing the right way to train a model. Initially, what seemed intuitive was the fact that increasing the amount of training sessions and epochs would improve the results. However, experiments quickly showed that excessive training does not necessarily mean a better result, but it can easily lead to a model that is heavily overfitted.

A. Training Configuration

The model was trained using batches of MRI images and their corresponding segmentation masks. The dataset was divided into training, validation, and test subsets. During each epoch, the model updated its parameters using gradient-based optimization in order to minimize the segmentation loss. The Adam optimizer [3] was selected for its adaptive learning rate properties and stable convergence behavior.

Validation metrics were monitored throughout training to evaluate the model's ability to generalize beyond the training data. The best-performing model checkpoint was saved at the epoch with the highest validation Dice Score, and later used for inference in the deployed application. The baseline training configuration consisted of a learning rate of 1×10^{-4} , a batch size of 4, and 20 epochs, running on a CUDA-enabled GPU.

B. Augmentation

To increase the diversity of the training data, several augmentation techniques were applied exclusively to the training set. These transformations included horizontal flips (applied with 50% probability), vertical flips (30% probability), random rotations of 90°, 180°, or 270° (50% probability), brightness adjustments of up to $\pm 20\%$ (50% probability), and Gaussian noise with a standard deviation of 0.02 (30% probability). All geometric transformations were applied identically to both the image and its corresponding mask to preserve label consistency.

Data augmentation artificially generates new training samples from existing images, helping the model become more robust to variations that may occur in real-world MRI scans. As a result, the risk of overfitting is reduced and the model is encouraged to learn more generalizable features.

V. RESULTS

Table I summarizes the performance of each training run. Four experiments were conducted in total. The higher learning rate of 10^{-3} led to the worst convergence, suggesting that smaller gradient steps are more appropriate for this task. The augmented run at 20 epochs performed slightly below the baseline, which is expected — augmented data introduces more variability per epoch and therefore requires more time to converge. Extending the augmented training to 40 epochs confirmed this hypothesis, achieving the best overall result with a validation Dice Score of **0.8569** and IoU of **0.7629**, surpassing the baseline by 1.4 percentage points.

TABLE I
MODEL PERFORMANCE ACROSS EXPERIMENTS

Run	LR	Aug.	Epochs	Val Dice	Val IoU
Baseline	10^{-4}	No	20	0.8431	0.7460
Higher LR	10^{-3}	No	20	0.8299	0.7311
Augmented 20ep	10^{-4}	Yes	20	0.8382	0.7404
Augmented 40ep	10^{-4}	Yes	40	0.8569	0.7629

Fig. 1 shows the training and validation curves for the baseline run. The loss decreases steadily throughout training, and both the Dice Score and IoU improve consistently, with no significant divergence between training and validation curves, which indicates the model is generalizing well without overfitting.

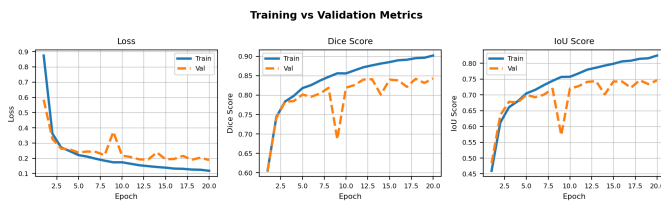


Fig. 1. Training vs. validation Loss, Dice Score, and IoU — Baseline (LR= 10^{-4} , no augmentation).

Fig. 2 shows the same metrics for the augmented 20-epoch run. Compared to the baseline, the curves show slightly slower convergence in the early epochs, which is expected when augmentation is applied, as the model encounters more diverse inputs per epoch.

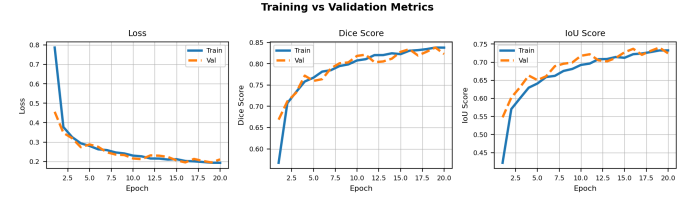


Fig. 2. Training vs. validation Loss, Dice Score, and IoU — Augmented, 20 epochs (LR= 10^{-4}).

Fig. 3 shows the augmented run extended to 40 epochs. The validation Dice Score continues to improve up to epoch 39, confirming that augmentation requires more epochs to reach its full potential. This run achieved the best overall result with a validation Dice Score of 0.8569.

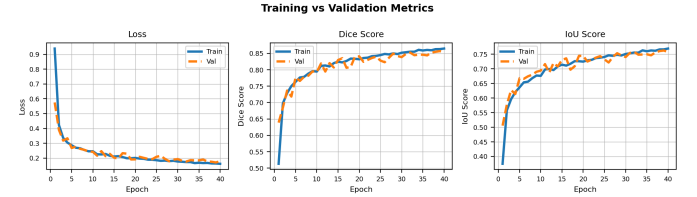


Fig. 3. Training vs. validation Loss, Dice Score, and IoU — Augmented, 40 epochs (LR= 10^{-4}). Best overall result.

Fig. 4 provides a direct comparison of all four runs. The augmented 40-epoch run achieves the highest validation Dice Score, while the higher learning rate run converges to the lowest performance. The results confirm that augmentation combined with sufficient training epochs yields the best generalization.

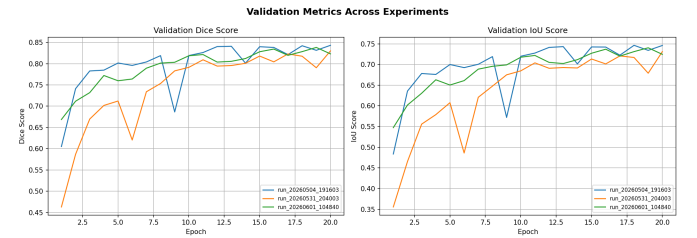


Fig. 4. Validation Dice Score and IoU across all training experiments.

VI. INTERFACE

The Streamlit interface allows users to upload the MRI image, and receive instant segmentation results. The result given displays the original scan, the predicted binary mask, as well as a color overlay that highlights the detected tumor region. The estimated tumor area is also computed from the

predicted mask and displayed as a percentage of the total image, providing a simple quantitative indicator of tumor extent that can be useful for a quick visual assessment.

VII. CONCLUSIONS

Throughout working on this project, I gained a lot of knowledge regarding how machine learning works in practice. From having to understand the importance of the dataset and how to preprocess the data, all the way to trying different ways of training the model in order to end up with a better result, it all has helped me to understand the fundamentals of this subset of Artificial Intelligence on a deeper level.

The experiments showed that a lower learning rate is more suitable for this task, and that data augmentation is a promising direction that would likely yield better results with a higher epoch count. Future work could explore training with more epochs on the augmented dataset, applying automated hyperparameter optimization to find the optimal configuration systematically, and extending the model to produce multi-class segmentation masks that distinguish between different tumor types or grades.

REFERENCES

- [1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.
- [2] BRISC2025 Dataset, Kaggle. [Online]. Available: <https://www.kaggle.com>
- [3] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [4] Streamlit. [Online]. Available: <https://streamlit.io>